

KREACIJSKI DIZAJN PATERNI

Singleton

Uloga Singleton paterna je da osigura da se klasa može instancirati samo jednom i da osigura globalni pristup kreiranoj instanci. Za praćenje rada sistema, objekat koji je potrebno samo jednom instancirati i nad kojim je potrebna jedinstvena kontrola pristupa jeste DataContext – centralni pristupnik ka bazi podataka koji sadrži sve kolekcije podataka sistema.

To možemo implementirati:

- kreiranjem DataContext klase s privatnim konstruktorom;
- definiranjem privatne statičke varijable koja čuva jedinu instancu;
- dodavanjem javne statičke metode GetInstance() koja vraća tu instancu;

Korištenje:

Unutar sistema Singleton se koristi tako što svaki kontroler, umjesto da sam kreira DataContext objekat, poziva DataContext.GetInstance(). Na taj način svi dijelovi sistema uvijek rade nad istim skupom podataka, transakcije koje obuhvataju više tabela ostaju konzistentne, a broj otvorenih konekcija prema bazi podataka ostaje pod kontrolom.

Prototype

Prototype patern obezbjeđuje interfejs za konstrukciju objekata kloniranjem jedne od postojećih prototip instanci i modifikacijom te kopije. Ukoliko bi korisničke klase proširili dodatnim atributima koji su u većini slučajeva isti za mnoge pojedince, imalo bi smisla kreirati modele s predefinisanim poljima koja se često ponavljaju. Kreiranje novih instanci bi se tada svelo na kloniranje tih prototipova te eventualno manje izmjene.

To možemo implementirati:

- definiranjem prototipova korisnika (Korisnik, Doktor, MedicinskaSestra) s podrazumijevanim postavkama;
- dodavanjem metode Clone() u korisničke klase koja kreira kopiju objekta;
- kloniranjem odgovarajućeg prototipa pri kreiranju novog korisnika te izmjenom samo specifičnih polja.

Korištenje:

Prototype patern se može koristiti za kreiranje korisničkih naloga na osnovu prethodno definisanih prototipova. Svaka uloga bi imala unaprijed postavljena prava pristupa, podešavanja i strukturu naloga. Prilikom dodavanja novog korisnika, sistem bi klonirao odgovarajući prototip i izvršio manje izmjene poput imena ili specijalizacije. Na ovaj način pojednostavljuje se proces kreiranja korisnika i osigurava konzistentnost između naloga iste vrste.

Factory Method

Factory Method patern omogućava kreiranje objekata na način da podklase, na osnovu informacije od strane klijenta ili tekućeg stanja, odluče koju klasu instancirati. Ovaj obrazac može naći primjenu u sistemu HealthZone pri kreiranju različitih vrsta korisnika, odnosno pacijenata, doktora, medicinskih sestara/braće i administratora.

To možemo implementirati:

- definiranjem interfejsa IKorisnikFactory s metodom KreirajKorisnika();
- definiranjem konkretnih fabrika za svaki tip korisnika koje implementuju IKorisnikFactory;
- pozivanjem fabrike unutar KorisnikController-a pri registraciji, umjesto direktnog poziva konstruktora.

Korištenje:

Na primjer, prilikom kreiranja pacijenta, doktora ili administratora, klijent ne mora voditi računa o tome iz koje konkretno klase će biti instanciran objekat. Time se uklanjaju grananja u raznim dijelovima koda, odnosno logika odlučivanja se smješta na jedno mjesto.

Abstract Factory

Abstract Factory patern omogućava da se kreiraju porodice povezanih objekata bez specificiranja konkretnih klasa. Koristi se kada postoji hijerarhija objekata koja enkapsulira različite platforme sastavljene od grupe međusobno zavisnih objekata.

To možemo implementirati:

- definiranjem apstraktne fabrike koja sadrži interfejs za kreiranje svake vrste objekta iz porodice;
- definiranjem konkretnih fabrika koje implementuju apstraktnu fabriku za svaku varijantu;

- korištenjem odgovarajuće konkretne fabrike u klijentskom kodu bez poznavanja konkretnih klasa.

Korištenje:

Razmatrajući sistem HealthZone, Abstract Factory bi imao smisla u slučaju da sistem podržava više varijanti poliklinike – na primjer, privatnu i javnu, gdje svaka ima različite varijante korisnika i usluga. S obzirom na to da je sistem fokusiran na jedan tip poliklinike bez takvih varijacija, kompleksnost koju nosi Abstract Factory patern nije opravdana.

Builder

Builder dizajn patern je koristan za kreiranje složenih objekata korak po korak. Primjenjuje se kada objekat ima mnogo opcionalnih parametara ili kada je potrebna fleksibilnost u kreiranju objekta. U našem sistemu, primjer bi bila klasa ZahtjevZaOpremu koja sadrži obavezna polja (opis, vrsta zahtjeva) ali i opcionalna (hitnost, kategorija opreme, naziv).

To možemo implementirati:

- kreiranjem Builder klase koja prima obavezne parametre u konstruktoru;
- dodavanjem metoda za svako opcionalno polje koje vraćaju isti Builder objekat radi ulančavanja poziva;
- dodavanjem metode Build() koja na kraju kreira i vraća gotov objekat.

Korištenje:

Builder patern bi se mogao koristiti pri kreiranju zahtjeva za opremom ili medicinskog nalaza, gdje doktor postepeno popunjava podatke – prvo obavezna polja (vrsta zahtjeva, opis), a zatim po potrebi dodaje hitnost ili kategoriju opreme. Patern omogućava da se ti podaci dodaju samo kad su relevantni, bez komplikovanja osnovne strukture objekta.